

INHERITED FROM Helix Constitution

This module is a submodule of a project that includes the Helix Constitution submodule. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references apply unconditionally. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution#AGENTS.md> - LLMProvider Module > **Base agent rules:** `HelixConstitution/AGENTS.md` "READ IT FIRST." > All rules in `HelixConstitution/AGENTS.md` apply unconditionally. > Rules below extend them and MUST NOT weaken any inherited clause.

Module Overview

`digital.vasic.llmprovider` is a generic, reusable Go module providing LLM provider abstractions and utilities. It defines the core `LLMProvider` interface and common patterns for building LLM provider implementations, including circuit breakers, health monitoring, retry logic, and lazy loading. The module is designed for AI/LLM applications that need to integrate multiple LLM providers with fault tolerance and observability.

Module path: `digital.vasic.llmprovider` **Go version:** 1.25.3+ **Dependencies:** `digital.vasic.models`, `github.com/sirupsen/logrus` **Test Dependencies:** `github.com/stretchr/testify`

Package Responsibilities

Package	Path	Responsibility
<code>llmprovider</code>	<code>./</code>	Core types: <code>LLMProvider</code> interface, circuit breaker, health monitor, retry config, lazy provider, and associated utilities. This is the only package.

Dependency Graph

```
digital.vasic.models
  â†“
digital.vasic.llmprovider
  â†“
github.com/sirupsen/logrus
```

Key Files

File	Purpose
<code>provider.go</code>	LLMProvider interface definition with <code>Complete</code> , <code>CompleteStream</code> , <code>HealthCheck</code> , <code>GetCapabilities</code> , <code>ValidateConfig</code> methods
<code>circuit_breaker.go</code>	Circuit breaker implementation (503 lines) with closed/open/half-open states, failure counting, timeout

File	Purpose
health_monitor.go	Health monitoring (430 lines) with configurable thresholds, intervals, status transitions
retry.go	Retry logic (226 lines) with exponential backoff, jitter, HTTP status code detection
types.go	Empty file (types moved to models module)
circuit_breaker_test.go	Circuit breaker tests (585 lines)
health_monitor_test.go	Health monitor tests
retry_test.go	Retry logic tests
types_test.go	Empty test file
go.mod	Module definition and dependencies
CLAUDE.md	AI coding assistant instructions
README.md	User-facing documentation with quick start

Agent Coordination Guide

Division of Work

When multiple agents work on this module simultaneously, divide work by component categories:

1. **Interface Agent** â€œ Owns `LLMProvider` interface definition and related methods. Changes affect all provider implementations.
2. **Circuit Breaker Agent** â€œ Owns circuit breaker implementation and `CircuitBreakerManager`. Affects fault tolerance.
3. **Health Monitor Agent** â€œ Owns health monitoring, thresholds, status transitions.
4. **Retry Logic Agent** â€œ Owns retry configuration, exponential backoff, HTTP status detection.
5. **Lazy Provider Agent** â€œ Owns lazy initialization pattern and deferred provider creation.

Coordination Rules

- `LLMProvider` **interface changes** require coordination with all agents as this is the foundational interface.
- **Circuit breaker behavior changes** affect fault tolerance across all providers.
- **Health monitor thresholds** affect provider health detection and automatic failover.
- **Retry logic changes** affect error recovery and rate limiting handling.
- **Lazy provider changes** affect initialization patterns and startup performance.

Safe Parallel Changes

These changes can be made simultaneously without coordination: - Adding new methods to existing structs (if they donâ€™t affect interface) - Adding new configuration options with sensible defaults - Adding new helper functions - Updating documentation - Adding new test cases - Improving error messages or logging

Changes Requiring Coordination

- Modifying `LLMProvider` interface method signatures (breaks all implementations)
- Changing circuit breaker state machine logic (affects fault tolerance)

- Modifying health monitor status transition thresholds (affects provider health detection)
- Changing retry exponential backoff formula (affects error recovery timing)
- Modifying lazy provider initialization semantics (affects startup behavior)

Build and Test Commands

```
# Build all packages
go build ./...

# Run all tests with race detection
go test ./... -count=1 -race

# Run unit tests only (short mode)
go test ./... -short

# Run a specific test
go test -v -run TestCircuitBreaker ./...

# Format code
gofmt -w .

# Vet code
go vet ./...

# Check dependencies
go mod tidy
go mod verify
```

Commit Conventions

Follow Conventional Commits with llmprovider scope:

```
feat(llmprovider): add new method to LLMProvider interface for batch process
feat(circuit): add configurable failure threshold to circuit breaker
feat(health): add degraded state to health monitor
feat(retry): add jitter to exponential backoff
fix(circuit): correct race condition in state transition
test(health): add edge case tests for health monitor thresholds
docs(llmprovider): update API reference for new methods
refactor(retry): extract common backoff calculation functions
```

Thread Safety and Concurrency

- `CircuitBreaker` uses `sync.RWMutex` for thread-safe state management
- `HealthMonitor` uses atomic operations for status updates
- `CircuitBreakerManager` is thread-safe for concurrent provider management
- `RetryConfig` is immutable after creation
- `LazyProvider` uses `sync.Once` for thread-safe lazy initialization
- All exported methods are safe for concurrent use unless otherwise documented

Integration Notes

- This module depends on `digital.vasic.models` for `LLMRequest`, `LLMResponse`, `ProviderCapabilities` types
- Provider implementations should import this module and implement the `LLMProvider` interface
- The module is designed for zero-dependency provider implementations (implementations only need this module and models)
- Circuit breaker and health monitor can be composed around any `LLMProvider` implementation